Adriel Barbosa de Freitas – 1250115728
Camile Pimenta Monteiro – 1240201525
Daniel Salgueiro Puga Paiva de Andrade – 1240206049
Fabio Cezar Paiva Paixão – 1240113895
Gustavo Pereira Fernandes Souto – 1240111496
Letícia dos Reis Prado – 1240207303

OcupaSala- Monitor de ocupação das salas de estudo da biblioteca UVA

Rio de janeiro

2026

Universidade
Veiga de Almeida

1) Resumo do que foi desenvolvido

- Sistema inteligente de monitoramento e controle de lotação em tempo real para ambientes fechados.
- Arquitetura completa desenvolvida: leitura de sensores de passagem, sinalização audiovisual com LED bicolor e buzzer, interface serial/Bluetooth protegida por divisor de tensão (3,3V) e alimentação autônoma por bateria de 9V.
- Toda a lógica, tratamento de bordas e limites operacionais foram validados em ambiente de simulação virtual.

2) Hardware montado até o momento

- Ambiente de Montagem: Protótipo virtual montado e conectado em protoboard completa no Autodesk Tinkercad.

- Componentes interligados:
- Microcontrolador principal (Arduino Uno).
- 2 Sensores digitais de presença/passagem (PIR/IR) alocados nas portas digitais D2 (entrada) e D3 (saída).
- Sinalizadores luminosos: 1 LED Verde (porta D4) e 1 LED Vermelho (porta D5) com resistores limitadores de 220 Ω.
- 1 Buzzer piezoelétrico ativo (porta D6) para feedback sonoro e alarme.
- Módulo Bluetooth HC-05 (TX em D10; RX em D11) com divisor de tensão resistivo (1 kΩ e 2 kΩ) para rebaixamento seguro de 5V para 3,3V.
- Bateria de 9V conectada aos pinos VIN e GND para operação autônoma.

3) Sensores utilizados e testes realizados

- Sensores Utilizados: Sensores digitais de detecção de fluxo infravermelho/presença.

- Testes Realizados:
    - Detecção de Entrada e Borda de Subida: Comprovação de que uma pessoa mantida na frente do sensor incrementa a contagem apenas uma vez (0/5 -> 1/5).
    - Detecção de Saída: Comprovação do decremento correto ao liberar a sala (1/5 -> 0/5).
    - Proteção contra Contagem Negativa: Disparo do sensor de saída com a sala em 0/5, confirmando que a contagem não assume valores negativos.
    - Saturação e Bloqueio de Lotação: Entrada de 5 pessoas consecutivas com disparo do alarme sonoro, acendimento do LED vermelho e emissão de 3 bipes de rejeição na tentativa da 6ª entrada (Bloqueado: Sala cheia).

4) Código desenvolvido no Arduino Uno

```
#include <SoftwareSerial.h>

// Pinos
const int PINO_IR_ENTRADA = 2;
const int PINO_IR_SAIDA   = 3;
const int PINO_LED_VERDE  = 4;
const int PINO_LED_VERM   = 5;
const int PINO_BUZZER     = 6;
const int PINO_BT_RX      = 10;
const int PINO_BT_TX      = 11;

SoftwareSerial bluetooth(PINO_BT_RX, PINO_BT_TX);

// Variaveis de controle
const int CAPACIDADE_MAXIMA = 5;
int ocupacaoAtual = 0;
int estadoAnteriorEntrada = LOW;
int estadoAnteriorSaida   = LOW;

void setup() {
  Serial.begin(9600);
  bluetooth.begin(9600);

  pinMode(PINO_IR_ENTRADA, INPUT);
  pinMode(PINO_IR_SAIDA, INPUT);
  pinMode(PINO_LED_VERDE, OUTPUT);
  pinMode(PINO_LED_VERM, OUTPUT);
  pinMode(PINO_BUZZER, OUTPUT);

  atualizarIndicadores();

  Serial.println("Sistema iniciado");
  bluetooth.println("Sistema iniciado");
}

void loop() {
  int leituraEntrada = digitalRead(PINO_IR_ENTRADA);
  int leituraSaida   = digitalRead(PINO_IR_SAIDA);

  // Entrada de pessoa
  if (leituraEntrada == HIGH && estadoAnteriorEntrada == LOW) {
    if (ocupacaoAtual < CAPACIDADE_MAXIMA) {
      ocupacaoAtual++;
      enviarStatus("Entrada detectada");
    } else {
      enviarStatus("Bloqueado: Sala cheia");
```

```
      emitirAlertaBloqueio();
    }
    atualizarIndicadores();
    delay(200); // Evita leitura dupla
  }

  // Saida de pessoa
  if (leituraSaida == HIGH && estadoAnteriorSaida == LOW) {
    if (ocupacaoAtual > 0) {
      ocupacaoAtual--;
      enviarStatus("Saida detectada");
    }
    atualizarIndicadores();
    delay(200); // Evita leitura dupla
  }

  estadoAnteriorEntrada = leituraEntrada;
  estadoAnteriorSaida   = leituraSaida;
}

// Atualiza LEDs e buzzer
void atualizarIndicadores() {
  if (ocupacaoAtual >= CAPACIDADE_MAXIMA) {
    digitalWrite(PINO_LED_VERDE, LOW);
    digitalWrite(PINO_LED_VERM, HIGH);
    digitalWrite(PINO_BUZZER, HIGH);
  } else {
    digitalWrite(PINO_LED_VERDE, HIGH);
    digitalWrite(PINO_LED_VERM, LOW);
    digitalWrite(PINO_BUZZER, LOW);
  }
}

// Alerta de sala cheia (3 bipes)
void emitirAlertaBloqueio() {
  for (int i = 0; i < 3; i++) {
    digitalWrite(PINO_BUZZER, HIGH);
    delay(100);
    digitalWrite(PINO_BUZZER, LOW);
    delay(100);
  }
}

// Envia dados para o monitor serial e bluetooth
void enviarStatus(String mensagem) {
  String logMsg = mensagem + " | Ocupacao: " + String(ocupacaoAtual) + "/" +
String(CAPACIDADE_MAXIMA);
  Serial.println(logMsg);
  bluetooth.println(logMsg);
}
```

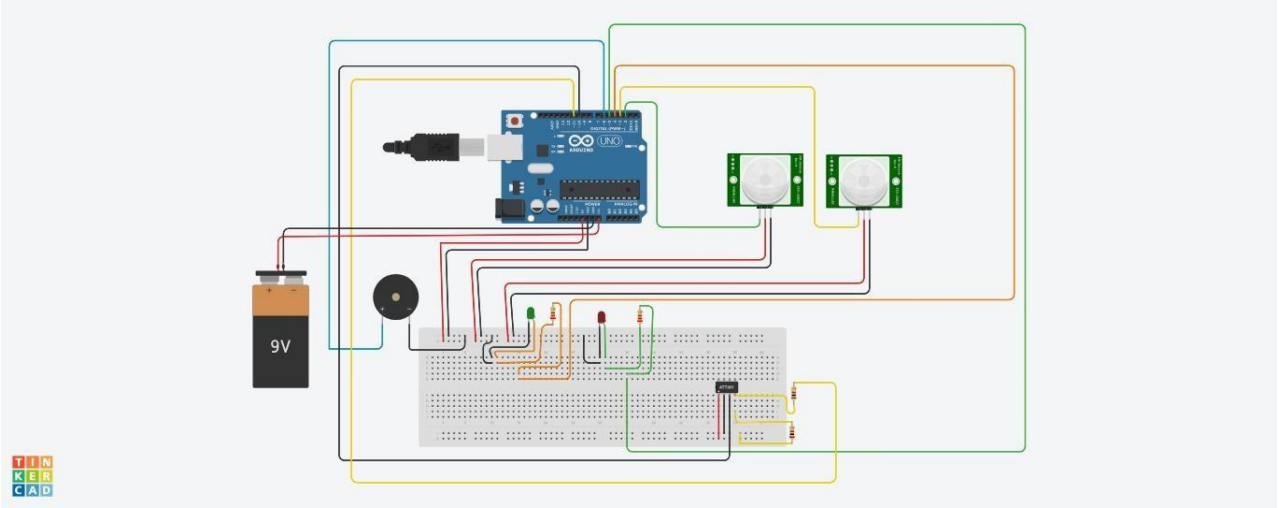
5) Descrição da lógica do programa

- Controle por Transição de Estados: O programa utiliza lógica de borda de subida (leitura == HIGH && anterior == LOW), evitando disparos falsos ou contagens repetidas.
- Máquina de Decisão:
    - Se entrada detectada e ocupação < limite: incrementa e atualiza LEDs.
    - Se entrada detectada e ocupação == limite: bloqueia, não incrementa e executa rotina emitirAlertaBloqueio().
    - Se saída detectada e ocupação > 0: decrementa e restaura o status verde se a lotação baixar de 5.

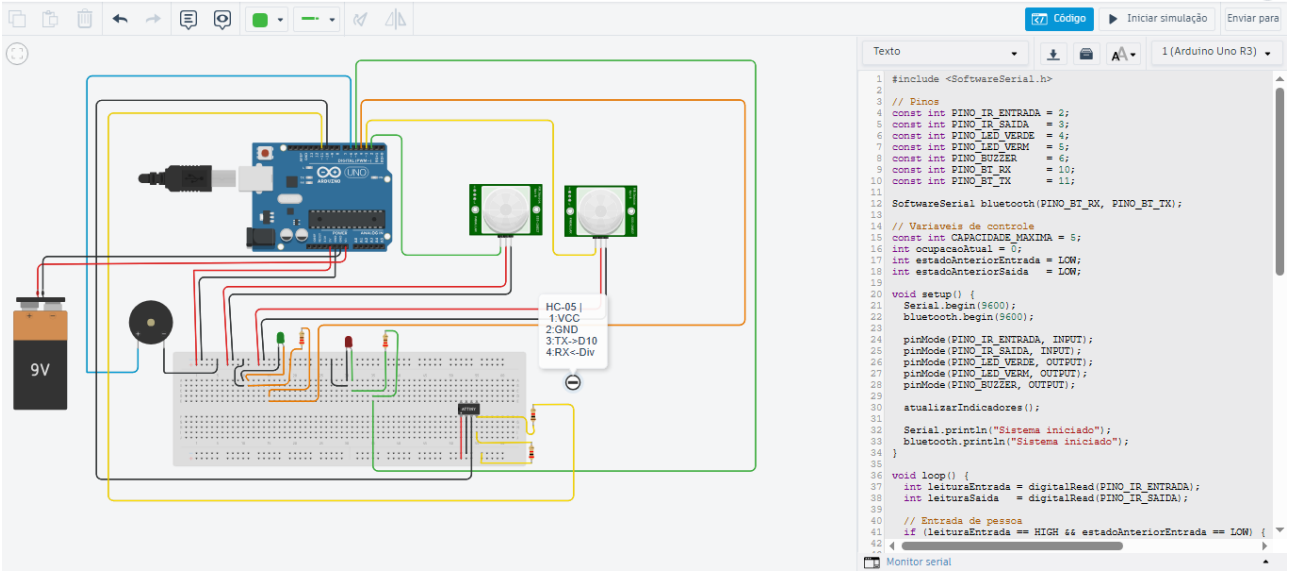
- Telemetria Dupla: Cada evento transmite a string formatada simultaneamente para o Monitor Serial USB e para o canal serial do Bluetooth.

6) Evidências dos testes

- Foto do protótipo:

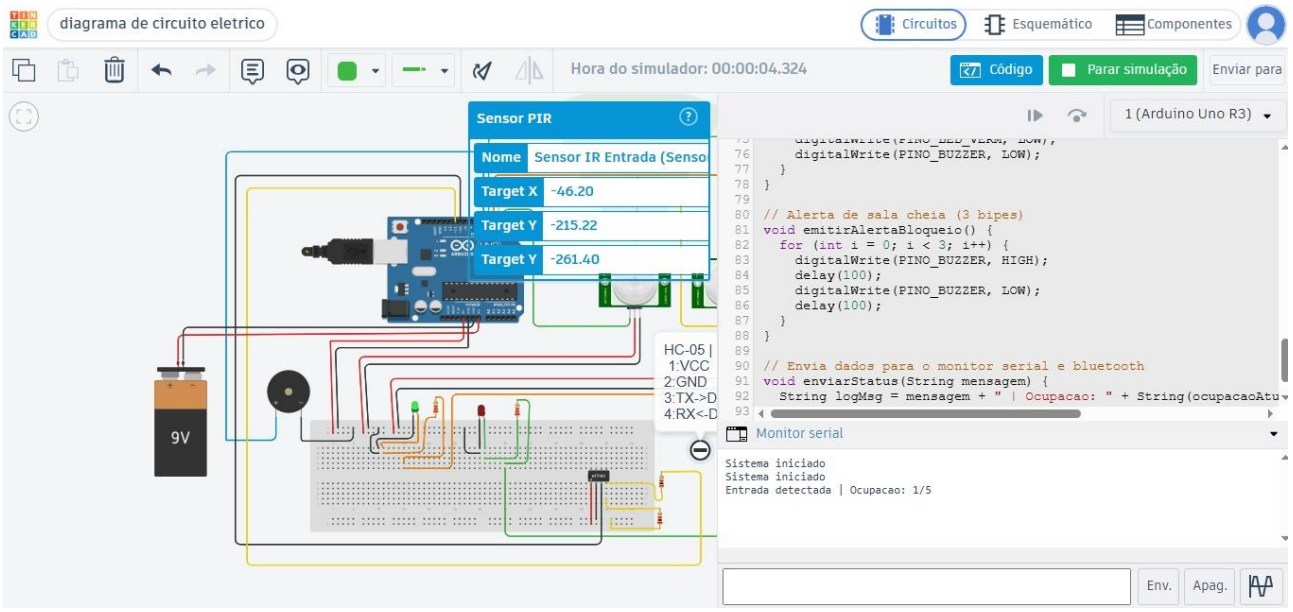


- Print do código:

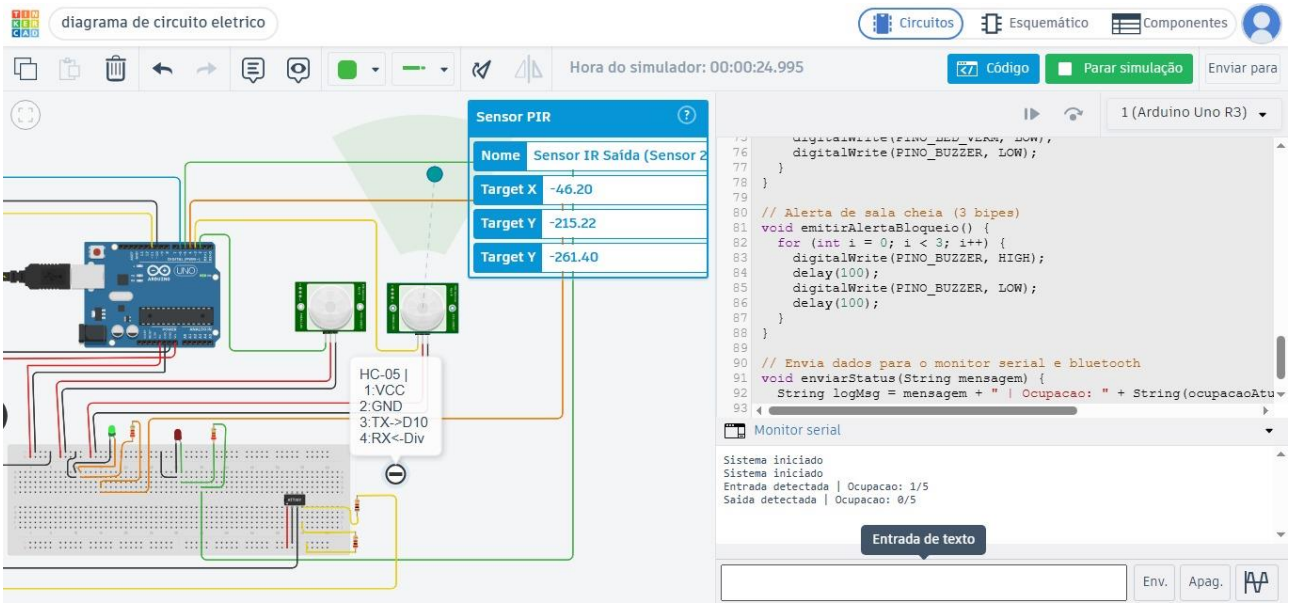


Prints do monitor serial:

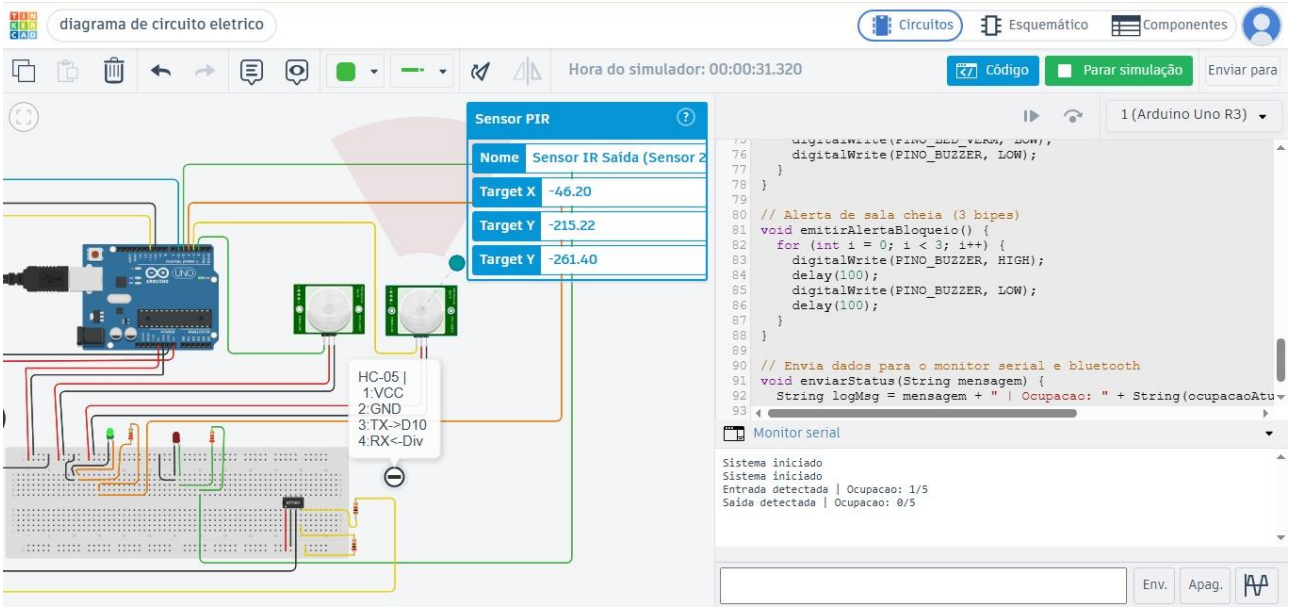
Teste 1:



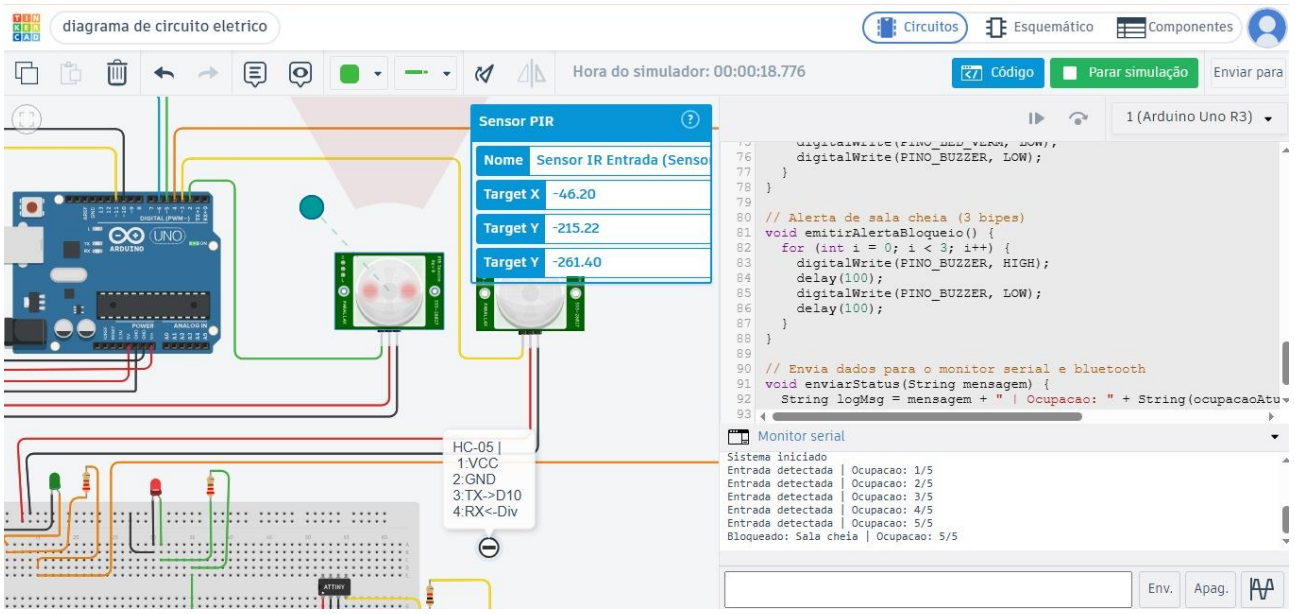
Teste 2:



Teste 3:



Teste 4:



7) Resultados obtidos

- Comportamento lógico 100% conforme os requisitos de projeto (resposta imediata dos atuadores, contagem estável e bloqueio seguro).
- Circuito de proteção de sinal validado matematicamente e funcionalmente para a recepção serial de 3,3V.

8) Problemas encontrados

- Atraso na entrega dos componentes físicos: Os módulos de hardware adquiridos ainda estão em trânsito logístico, impedindo o teste direto na bancada física.

- Incompatibilidade de nível de tensão (5V vs. 3,3V): Risco de sobretensão no pino RX do módulo Bluetooth/ESP32 a partir de saídas de 5V.
- Risco de disparos múltiplos nos sensores: Problema comum de oscilação contínua na leitura digital do sensor enquanto o usuário transita.

9) Como a equipe resolveu ou pretende resolver

- Resolução do atraso: Desenvolvimento de um gêmeo digital (*digital twin*) completo no Tinkercad para antecipar toda a engenharia de software e testes elétricos.
- Resolução da sobretensão: Dimensionamento e montagem de um divisor resistivo (1 kΩ / 2 kΩ) reduzindo a tensão para 3,33V.
- Resolução de disparos múltiplos: Implementação de variáveis de estado anterior e *debounce* por temporização no algoritmo em C++.

10) Próximas etapas até 21/09

- Recebimento e inspeção dos componentes físicos.
- Montagem do circuito real em bancada na protoboard física.
- Gravação e compilação do código final diretamente na placa microcontrolada física.